

malloc练习

1.checkpoint

1. 带边界的隐式自由列表
2. 标签合并。每个块都有表单的页眉和页脚。
3. 一个字为四个字节，双字为8个字节

2.内存初始化模块

1. 进行初始块分配；将虚拟地址空间映射到内存
2. 对齐填充，为了解决伪共享问题。
3. 确定序言块头部；转换DSIZE的类型。
4. 指向序言块；为了合并相邻空间
5. 它先使用mem_sbrk函数来分配4个字的空间，这些空间将被用作堆的初始空间，然后设置了前导块和后导块用来标记堆的开始和结束，最后使用extend_heap函数来扩展堆创建一个空闲块

3.内存分配模块

1. 用需求大小+头尾大小 向上取整;为了符合开销和对齐要求
2. 找不到合适的。获取更多内存并放置块；find_fit函数:提供首次匹配和最佳匹配两种实现。place函数用于返回bp的状态
3. 请求扩展内存大小；为了开辟更多空间以免数据丢失。
4. mm_malloc()函数用来分配大小为size的块。函数必须调整请求的大小，从而为头部和脚部留出空间，并满足双字对齐的要求。最小块大小为16字节(载荷不能为0)。分配器调整了请求大小之后，会搜索空闲链表，寻找一个合适的空闲块。如果能找到，就放这个请求块，并分割出多余的部分；如果不能找到，就调用extend_heap()来扩展堆，并将请求放在这个新空闲块中。最后返回新分配的块可选地分割：将请求块放置在空闲块的起始位置，只有当剩余部分的大于等于最小块大小时，才进行分割。

4.内存释放模块

1. 初始化bp的地址空间大小；GET_SIZE的作用是接受参数bp返回它的无符号整数的值与~0x7的位与结果。HDRP接受一个参数bp，返回它减去WSIZE后的地址。
2. 设置块的头部和尾部；为了与15-213malloc驱动程序保持一致
3. 合并相邻块；该函数依次参考各参数表达式，遇到非null值即停止并返回该值。如果所有的表达式都是空值，最终将返回一个空值。
4. mm_free()用于释放一个块。该函数释放所请求的块(bp)，并调用coalesce()函数来合并与之邻接的空闲块

5.堆扩展模块

1. 为了保证双字对齐；为了分配偶数个单词保证对齐。
2. 初始化自由块页眉/页脚和尾声页眉；为了方便合并
3. 如果前一个块空闲则合并。初始化创建空闲链表
4. 返回bp非零的值。返回列表中第一个非零的值。
5. extend_heap()为保持双字对齐，将请求大小向上舍入到最接近的2字倍数，然后调用mem_sbrk()请求额外的堆空间。该空间紧接在结尾块之后。最后，由于extend_heap()也可能被mm_malloc()调用，因此很可能旧的块是以一个空闲块结束，因此需要coalesce()函数来合并这两个空闲块。

6.块放置模块

1. 将asize字节的块放在空闲块bp的开头*并且如果余数至少是最小块大小，则进行拆分。
2. 对列表的头和尾进行初始化
3. 对下一个表的bp的头和尾进行初始化，方便链接。

7.查找模块

1. 为具有asize字节的块查找适合项,循环进行拟合搜索
2. 检查块是否已分配以及asize是否小于等于块的大小。为了防止系统出现内存分配问题

8.合并空闲块模块

1. 边界标记合并。将ptr返回到合并块